Deep Dive: Finding StyleGAN Generated Faces

Bryson R. Payne, Chase D. Manning
University of North Georgia
Department of Computer Science and Information Systems

ABSTRACT

With significant funding being funneled into content generating AI systems there is an ever-growing concern about the origins of digital content. Since the release of the advent of Stable Diffusion and GAN based models many artists and public figures have been on the attack against AI content generating systems that they feel are intruding in their career space or being used with malintent. With the incomprehensibly large datasets used to train these AI models they can generate content and deepfakes that are nearly indistinguishable from content made by humans. There is increasing desire for tools to confirm the legitimacy of digital content, whether it is for validating an upcoming artists skill, a college professor checking the legitimacy of a student's paper, or a public figures fighting against backlash from a viral deepfake. The problem is identifying the AI generated content when it has become indistinguishable from the alternatives. Using Artificial Intelligence to fight back is the perfect solution to the content generation arms-race. Focusing on one model this research will demonstrate the creation of am image classifier to detecting deepfakes created by StyleGAN. With the image classifier the more easily identifiable features in an image will lead to a drastic increase in the classifier's accuracy in respect to the dataset it is trained on. The image classifier will be used to compare human faces against AI generated faces from a Nvidia StyleGAN dataset.

Introduction

Creation of the convolutional neural network begins with choosing the datasets by which it will be trained on while paying attention to potential imbalance in the datasets. From here normalizing the data in the datasets for consistency and converting the data into a format that can be input into the model is necessary. Once the data is prepared the model can be ran and the results can be interpreted with small changes to the model to optimize the model avoiding issues such as overfitting.

BACKGROUND

Recent years, have seen an increase in the ease-of-use for AI-based content generation leading to a proliferation of AI research. In a paper discussing methods of GAN image detection, Gragnaniello, Marra and Verdoliva [1], highlight the strengths of the detection methods in the spatial domain regarding convolution neural networks as being great at such tasks. Researchers at the *University Federico II of Naples* [5] compared the effectiveness of various trained Convolutional Neural Networks (CNNs) in their detection of GAN generated images using the GAN discriminator as the baseline for measuring improvements. There has also been advances in GAN image detection through focusing on individual facial features such as inconsistencies in the eyes 3] when used alongside deep-learning training. However, while this is applicable to this research, a team of researchers at NVIDIA published a paper highlighting improvements to the StyleGAN architecture which would remove GAN artifacts improving the image quality [2]. However, a method proposed by Zhang, Karaman and Chang at Columbia University for simulating GAN artifacts in images to detect GAN images. This combats the trend that CNNs suffer from where the training of the CNN requires a large dataset of images from a GAN that usually isn't available or known [4].

IMPLEMENTATION.

Normalization of the Dataset

Due to computational limits the total size of the dataset had to be lowered to 200,000 images balanced between 100,000 AI deepfakes (image in Figure 1c) and 100,000 human faces (image in Figure 1a). To train the model with a single static resolution, resizing of the images in the datasets was necessary, but an issue arose where the images in the datasets consisted of two differing resolutions causing the convolutional neural network to detect the compression artifacts instead of the image's features leading to the classifier incorrectly prediction the categories. To correct this the images resolutions widths and heights were multiplied together and then the greatest common denominator was found allowing the images to be resized to a common resolution of 89x109 pixels with equal distortion for all normalized images across the datasets (see Figure 1b and 1d).

Creating the Training Data

With the normalized data the training data had to be converted to a valid format for training the convolutional neural network. Each image was converted to an array. From here the arrays had their corresponding category associated being either "Human" or "AI" and where randomized. The final step in preparation of training the model was outputting the data in a format that could be input into the model generator for which the python library pickle was chosen.

Implementation of the Model Generator

The convolutional neural network was trained in Tensorflow using a sequential model. Using a loss-function built into Keras Binary, Cross-entropy was chosen to measure loss due to it being optimized for measuring loss for binary classifications. The use-case for the image classifier with only two categories worked well for this loss-function. For optimization, the built-in adam optimizer was chosen for its ability to compute faster with fewer parameters meeting the resource requirements for generating the model without the computer running out of memory.

Model Optimizations

With the computational limits in place and the size of the dataset each model would take between seven to fifteen hours depending on the number of epochs and the size of the batches used. The optimal model from those generated was created with a batch size of 200, with three epochs and a validation split of 30 percent (see Figure 2). The stack of the model's layers started with a Conv2D layer with a value of 64 and a window size of three by three. The next layer chosen was a rectified linear activation function for its relatively high performance. From here a MaxPooling2D layer was added with a pool size of two by two. Then after repeating the previous three layers a flatten layer was added to prep the data for the following dense layer. Then another activation layer using rectified linear was added. Next, a dense layer of one would be added which would be activated by the final activation layer using the sigmoid activation function. Finally, the model would be compiled and ran through the fitness function.

(a) (b) (c) (d)

Figure 1. The default resolution for the human images dataset [6] 178x218 pixels (a), the normalized resolution for the human images dataset in grayscale 89x109 pixels (b), the default resolution for the AI generated images dataset [7] 1024x1024 pixels (c), the normalized resolution for the AI generated images dataset 89x109 pixels (d).

```
Epoch 1/5
700/700 [==============================] - 5658s 8s/step - loss: 0.0713 - accuracy: 0.9692 - val_loss: 0.0110 - val_accuracy:
0.9966
Epoch 2/5
700/700 [==============================] - 5690s 8s/step - loss: 0.0072 - accuracy: 0.9975 - val_loss: 0.0064 - val_accuracy:
0.9980
Epoch 3/5
700/700 [==============================] - 5747s 8s/step - loss: 0.0040 - accuracy: 0.9987 - val_loss: 0.0031 - val_accuracy:
0.9990
Epoch 4/5
700/700 [==============================] - 5617s 8s/step - loss: 0.0025 - accuracy: 0.9992 - val_loss: 0.0027 - val_accuracy:
0.9990
Epoch 5/5
700/700 [==============================] - 6605s 9s/step - loss: 0.0025 - accuracy: 0.9991 - val_loss: 0.0032 - val_accuracy:
0.9989
```

Figure 2: The run time Tensorflow output showing the in-sample evaluations for loss and accuracy alongside the out of sample evaluations for loss and accuracy. For the best preforming model, the image classifier preformed well until later into the 4th epoch.

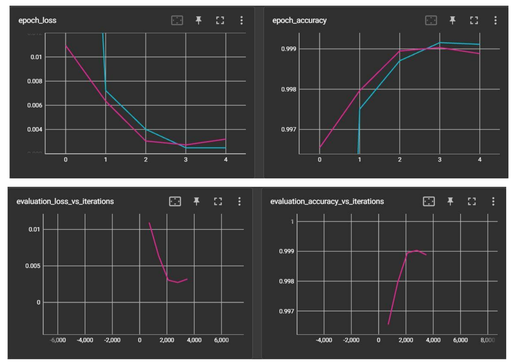

Figure 3a (top) A graphical representation of the model in Tensorboard showing its performance with within each epoch showing the point where overfitting started to occur. **3b** (bottom) Shows the graphical representation of the model during its evaluation throughout training and the point where accuracy and loss started getting work.

```
Model: "sequential"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 conv2d (Conv2D)             (None, 87, 107, 256)      2560

 activation (Activation)     (None, 87, 107, 256)      0

 max_pooling2d (MaxPooling2D  (None, 43, 53, 256)      0
 )

 conv2d_1 (Conv2D)           (None, 41, 51, 256)       590080

 activation_1 (Activation)   (None, 41, 51, 256)       0

 max_pooling2d_1 (MaxPooling  (None, 20, 25, 256)      0
 2D)

 flatten (Flatten)           (None, 128000)            0

 dense (Dense)               (None, 64)                8192064

 activation_2 (Activation)   (None, 64)                0

 dense_1 (Dense)             (None, 1)                 65

 activation_3 (Activation)   (None, 1)                 0

=================================================================
Total params: 8,784,769
Trainable params: 8,784,769
Non-trainable params: 0
```

Figure 4 A model summery of the final model including the model's type, the layers used, and the number of parameters created by each layer. The summery also shows the total number of parameters contrasted to the total number of trainable parameters.

RESULTS

After training, the final model testing of its accuracy with untrained images began and suspicions arose that an issue had occurred during training. The assertion was that the issue was connected to a fault in the dataset, considering the speed at which the model's accuracy and loss improved even within the first few epochs. Given the classification goals for this model compared to models trained to perform similar tasks it would be within expectations for at least 10 epochs to be required to reach a reasonable accuracy and loss. Testing of the model for the assumed issue began with scaling and cropping AI images to the dimensions of the other categories dataset. Feeding the AI images into the model revealed a consistent prediction of the Human category. The conclusion reached after the testing concluded was that the model had managed to detect differences related to the image sizing despite effort to circumvent this outcome. The process of up-sampling and down-sampling the images to standardize any compression artifacts did not accomplish the task of normalizing the dataset.

FUTURE WORK

Given the issues that arose during the creation of the model the research team would like to pursue additional routes of circumventing the image sizing issue with the dataset. Cropping images instead of up/down sampling may provide a more consistent normalization of the dataset.

REFERENCES

1. Lakshmanan Nataraj et al. 2019. Detecting gan generated fake images using co-occurrence matrices. Electronic Imaging 31, 5 (2019). DOI:http://dx.doi.org/10.2352/issn.2470-1173.2019.5.mwsf-532
2. Tero Karras, Samuli Laine, Miika Aittala, Janne Hellsten, Jaakko Lehtinen, and Timo Aila. 2020. Analyzing and improving the image quality of stylegan. (March 2020). Retrieved March 7, 2023 from https://arxiv.org/abs/1912.04958
3. Xin Wang, Hui Guo, Shu Hu, Ming-Ching Chang, and Siwei Lyu. 2023. Gan-generated faces detection: A survey and new perspectives. (January 2023). Retrieved March 7, 2023 from https://doi.org/10.48550/arXiv.2202.07145
4. Xu Zhang, Svebor Karaman, and Shih-Fu Chang. 2019. Detecting and simulating artifacts in gan fake images. 2019 IEEE International Workshop on Information Forensics and Security (WIFS) (2019). DOI:http://dx.doi.org/10.1109/wifs47025.2019.9035107
5. F. Marra, D. Gragnaniello, D. Cozzolino and L. Verdoliva, "Detection of GAN-Generated Fake Images over Social Networks," 2018 IEEE Conference on Multimedia Information Processing and Retrieval (MIPR), Miami, FL, USA, 2018, pp. 384-389, doi: 10.1109/MIPR.2018.00084.
6. Li, J. (2018, June 1). Celebfaces attributes (celeba) dataset. Kaggle. Retrieved March 20, 2023, from https://www.kaggle.com/datasets/jessicali9530/celeba-dataset
7. Reben, A. (2019, April 18). 1 million fake faces : Alexander Reben : Free download, Borrow, and streaming. Internet Archive. Retrieved March 20, 2023, from https://archive.org/details/1mFakeFaces